



LABORATORIO NO. 07

"Punteros y arreglos"

OBJETIVOS

- ✓ Aplicar los conocimientos sobre punteros.
- ✓ Utilizar un arreglo con punteros.
- ✓ Punteros a Funciones
- ✓ Punteros a arreglos de funciones

PRÁCTICA

1. Cree una nueva aplicación en C++ con la plantilla **Aplicación de consola**

a. **Nombre de Proyecto:** Lab7_<primer nombre>_<primer apellido>+_<carne> y resuelva

Se necesita desarrollar un sistema para gestionar los empleados de una empresa. El sistema debe permitir administrar diferentes tipos de empleados y generar reportes utilizando herencia, polimorfismo, punteros, memoria dinámica, arrays de punteros a funciones y punteros a funciones.

Requerimientos funcionales

1. Clase abstracta Empleado

Crear una clase abstracta Empleado con los atributos:

- nombre
- id
- salario

Debe incluir un método virtual puro:



mostrarInfo()

2. Clases derivadas

Crear las siguientes clases:

EmpleadoAdministrativo

- departamento
- bonificacion

EmpleadoTecnico

- especialidad
- horasExtra

EmpleadoVentas

- comision
- ventasRealizadas

Cada clase debe implementar mostrarInfo() de manera polimórfica.

3. Array dinámico de punteros

Los empleados deberán almacenarse en un:

Empleado** empleados;

utilizando memoria dinámica.

El programa debe utilizar correctamente new y delete.

4. Array de punteros a funciones para reportes

Crear un array de punteros a funciones que permita ejecutar:

- Mostrar todos los empleados.
- Calcular el salario total.
- Contar empleados por tipo.



- Mostrar empleados con salario mayor a un valor indicado.

El usuario deberá seleccionar el reporte desde un menú.

5. Operaciones con empleados

El sistema debe permitir:

- Agregar un empleado según su tipo.
- Buscar un empleado por id.
- Eliminar un empleado por id, liberando correctamente la memoria.
- Mostrar la información del empleado encontrado.

6. Actualizar salario mediante punteros a funciones

Implementar las siguientes funciones:

- `aumentarSalario()` → aumenta un porcentaje.
- `disminuirSalario()` → disminuye un porcentaje.
- `cambiarSalario()` → asigna un nuevo salario.

Estas operaciones deberán ejecutarse utilizando **punteros a funciones**.

7. Ordenar empleados

Permitir ordenar los empleados por:

- Salario ascendente.
- Salario descendente.
- Nombre alfabéticamente.

8. Estadísticas

El sistema deberá mostrar:

- Salario promedio.
- Empleado con mayor salario.
- Empleado con menor salario.
- Cantidad de empleados administrativos.
- Cantidad de empleados técnicos.
- Cantidad de empleados de ventas.



Menú sugerido

=====

GESTIÓN DE EMPLEADOS

=====

1. Agregar empleado
2. Mostrar empleados
3. Buscar empleado
4. Eliminar empleado
5. Actualizar salario
6. Ordenar empleados
7. Generar reportes
8. Mostrar estadísticas
9. Salir

Restricciones

- Utilizar una clase abstracta.
- Utilizar herencia y polimorfismo.
- Utilizar un array dinámico de punteros a Empleado.
- Utilizar new y delete.
- Utilizar un array de punteros a funciones para los reportes.
- Utilizar punteros a funciones para actualizar salarios.
- No utilizar vector.
- El destructor de Empleado debe ser virtual.

Clave de calificación

Criterio	Descripción	Puntos
POO y Polimorfismo	Clase abstracta Empleado	10
	Herencia y clases derivadas	10
	Polimorfismo mediante mostrarInfo()	10
Punteros y memoria dinámica	Array dinámico de punteros a Empleado	10
	Uso correcto de new y delete	10
	Agregar, buscar y eliminar empleados	5
Punteros a funciones	Array de punteros a funciones para reportes	10
	Ejecución de reportes mediante punteros a funciones	10
	Actualización de salario mediante punteros a funciones	5
Funcionalidades adicionales	Ordenamiento de empleados	5
	Estadísticas solicitadas	5
	Organización, validaciones y calidad del código	10
Total		100